State Based Testing

1. Introduction

State-Based Testing is a testing technique where the behavior of a system is verified based on its **states and transitions**.

It ensures that:

- Given an initial state
- When an event occurs
- The system transitions to the correct next state

---

2. Objective

- To verify **state transitions**
- To validate **system behavior**
- To ensure correctness of **state changes**
- To detect invalid transitions

---

3. Module Selected

Admin Authentication System (from your project)

Reason:

- Clearly has states

---

4. States Identification

State	Description
Idle	System waiting for login
EnterCredentials	User enters username/password
Authenticated	Correct login
Retry	Wrong credentials
Locked	Too many failed attempts

---

5. State Transition Table

Current State	Event	Next State
Idle	Enter credentials	EnterCredentials
EnterCredentials	Correct login	Authenticated
EnterCredentials	Wrong login	Retry
Retry	Correct login	Authenticated
Retry	Wrong (3rd attempt)	Locked
Locked	Any input	Locked

---

6. State Diagram

```
Idle → EnterCredentials → Authenticated
              ↓
            Retry → Locked
```

---

7. STATE-BASED TEST CASES (PDF STYLE)

◇ TC1: Initial State

- Initial State: Idle
- Action: Start system
- Expected: Idle

◇ TC2: Successful Login

- Initial State: Idle
- Input: Correct username + password
- Expected State: Authenticated

◇ TC3: One Wrong Attempt

- Initial State: EnterCredentials
- Input: Wrong password
- Expected State: Retry

◇ TC4: Correct After Retry

- Initial State: Retry
- Input: Correct credentials
- Expected State: Authenticated

◇ TC5: Three Wrong Attempts

- Initial State: EnterCredentials
- Input: Wrong × 3
- Expected State: Locked

◇ TC6: Locked State Behavior

- Initial State: Locked
- Input: Any credentials
- Expected: Remain Locked

---

8. TEST IMPLEMENTATION CODE (LIKE PDF STYLE)

This is exactly like PDF test class (converted to C++)

```
#include <iostream>
#include <string>
using namespace std;

class LoginSystem {
public:
    enum State {
        IDLE,
        ENTER,
        AUTHENTICATED,
        RETRY,
        LOCKED
    };
```

```
private:
    State currentState;
    int attempts;

public:
    LoginSystem() {
        currentState = IDLE;
        attempts = 0;
    }

    void enterCredentials(string user, string pass) {
        if(currentState == LOCKED) return;

        if(user == "parth" && pass == "1234") {
            currentState = AUTHENTICATED;
        } else {
            attempts++;
            if(attempts >= 3)
                currentState = LOCKED;
            else
                currentState = RETRY;
        }
    }

    State getState() {
        return currentState;
    }
};
```

```
//
==========================================================
===
//  TEST CASES (STATE-BASED)
//
==========================================================
===

int main() {

    cout << "=========== STATE BASED TESTING
===========\n";

    // TC1: Initial State
    LoginSystem sys1;
    if(sys1.getState() == LoginSystem::IDLE)
        cout << "TC1 Passed\n";

    // TC2: Successful Login
    LoginSystem sys2;
    sys2.enterCredentials("parth","1234");
    if(sys2.getState() == LoginSystem::AUTHENTICATED)
        cout << "TC2 Passed\n";

    // TC3: One Wrong Attempt
    LoginSystem sys3;
    sys3.enterCredentials("wrong","wrong");
    if(sys3.getState() == LoginSystem::RETRY)
```

```
        cout << "TC3 Passed\n";

    // TC4: Correct After Retry
    LoginSystem sys4;
    sys4.enterCredentials("wrong","wrong");
    sys4.enterCredentials("parth","1234");
    if(sys4.getState() == LoginSystem::AUTHENTICATED)
        cout << "TC4 Passed\n";

    // TC5: Three Wrong Attempts
    LoginSystem sys5;
    sys5.enterCredentials("a","b");
    sys5.enterCredentials("a","b");
    sys5.enterCredentials("a","b");
    if(sys5.getState() == LoginSystem::LOCKED)
        cout << "TC5 Passed\n";

    // TC6: Locked State
    LoginSystem sys6;
    sys6.enterCredentials("a","b");
    sys6.enterCredentials("a","b");
    sys6.enterCredentials("a","b");
    sys6.enterCredentials("parth","1234");
    if(sys6.getState() == LoginSystem::LOCKED)
        cout << "TC6 Passed\n";

    return 0;
}
```

---

9. EXPECTED OUTPUT

========== STATE BASED TESTING ==========

TC1 Passed
TC2 Passed
TC3 Passed
TC4 Passed
TC5 Passed
TC6 Passed

---

10. Observation

- All states correctly handled
- Transitions verified
- Invalid transitions blocked
- System behaves as expected

11. **CHECKLISTS**

(Based on Admin Authentication State Model)

---

1. Structure Checklist

Validation

- There is an **initial state (Idle)** with only outbound transitions
- There is a **final state (Locked)** with only inbound transitions

- No equivalent or duplicate states exist
- Every state is reachable from the initial state:
  - Idle → EnterCredentials → Retry → Locked
- Final state is reachable from all states:
  - From Retry → Locked
- Every event appears in at least one transition:
  - Enter credentials
  - Correct login
  - Wrong login
- Every state (except initial and final) has:
  - At least one incoming transition
  - At least one outgoing transition

Conclusion

The state model is structurally complete, with all states reachable and properly connected.

---

2. State Name Checklist

Validation

- All state names are **meaningful in context**:
  - Idle → waiting state
  - Authenticated → success state

    - Locked → failure state
- No unnecessary or redundant states
- No vague names like “State1” or “Temp”
- State names are **passive (adjectives/conditions)**:
    - Authenticated
    - Locked

Conclusion

All state names are meaningful, consistent, and clearly represent system conditions.

---

3. Guarded Transition Checklist

Validation

- Entire range of conditions is covered:
    - Correct credentials
    - Wrong credentials
- Guards are **mutually exclusive**:
    - Cannot be both correct and wrong at same time
- Guard variables are visible:
    - Username, Password
- No ambiguity in transitions:
    - Each input leads to exactly one state

- No side effects during guard evaluation:
  - Only checks conditions, no state corruption

Conclusion

All transitions are well-defined with mutually exclusive conditions and complete coverage of input cases.

---

4. Robustness Checklist

Validation

- Invalid inputs are handled:
  - Wrong credentials → Retry
- System prevents illegal transitions:
  - Locked state cannot go back
- System remains stable:
  - No crash on invalid input
- Error handling mechanism exists:
  - Retry attempts
- System preserves valid state:
  - Locked remains Locked
- **Conclusion:**
  The system is robust and handles invalid inputs safely without corrupting its state.